

Exam 4: MDPs and RL

Name:

CSC 480: Artificial Intelligence, Spring 2026

CalPoly Email:

1. (0.3 points) When modeling a problem as an MDP, we must assume the *Markov Property* where our transitions and rewards do not depend any previous states before the current state. Why is this assumption required in order for the Bellman Equation to hold?

The Bellman equation decomposes the value of a state as $V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$

This decomposition treats $V^*(s')$ as a self-contained quantity, ie the total expected future reward from s' forward. That only makes sense if s' is a *sufficient statistic* for all future behavior (so knowing s' is all you need to determine what happens next). If the Markov Property did not hold, the transition probabilities and rewards would depend on the full history $(s_0, a_0, s_1, a_1, \dots, s_t)$, not just the current state. In that case, $V^*(s')$ could not be defined as a function of s' alone, as two trajectories arriving at the same state s' via different histories could yield different future values, so there would be no single well-defined $V^*(s')$ to substitute into the right-hand side. The recursive structure of the Bellman equation collapses without this guarantee.

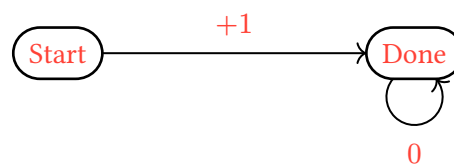
2. (0.3 points) For an arbitrary environment that may have so called “path dependence” or “memory”, where the path taken to reach a state may affect the environment at that state, is it *always* possible to formulate an equivalent MDP? If so, how? If not, why?

Yes. The trick is to *bake the history into the state*. If an environment has memory (ie the outcome depends on how you got there, not just where you are), we can define a new, bigger state that includes the relevant past. For example, if the environment only remembers the last two steps, define the state as a pair (s_{t-1}, s_t) instead of just s_t . Now the new state contains all the information the environment cares about, so the Markov Property holds and we have a valid MDP.

In the extreme case, the new state is the entire history of states and actions. This always works in principle, but the state space can grow very large (or infinite), making it expensive or impossible to solve in practice. The important thing is that by encoding history into the state definition, each state becomes independently evaluable thus satisfying the Markov Property by construction.

3. (0.3 points) Is it ever possible to calculate the optimal value function V^* for an *infinite horizon* MDP with discount factor $\gamma = 1$. If so, sketch an example. If not, explain why.

Yes, if the MDP has an absorbing terminal state that every policy reaches with probability 1. Once the agent enters a terminal state it collects no further reward, so the infinite sum is still finite even with $\gamma = 1$.



The problem with $\gamma = 1$ in general is that a policy which never terminates (staying in a rewarding state forever or exploring infinite rewarding states) produces an infinite sum with no meaningful value. Since γ is one, a non-zero infinite series of rewards will never converge, meaning an unrewarding terminal structure is the only way to get a finite value.

4. (0.6 points) Consider an MDP inhabited not only by your agent the robot wizard, but also an evil goblin whose only goal is to minimize the wizard's reward. In this MDP the wizard plays a turn, and then the goblin plays a turn, and the two agents continue to alternate using the same transition function T . Let A be the set of actions available to the wizard, and B be the set of actions available to the goblin. The game has an infinite horizon, and uses a single discount factor γ for both agents. Let R be the utility received by the wizard.

(a) (0.3 points) If we assume both the wizard and the goblin are rational and act optimally according to their adversarial goals, which, if any, of the following expressions correctly applies the Bellman equation to update Q values of wizard actions ($Q_w^*(s, a)$ being the value of the wizard taking action a in state s).

☐ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \min_{a' \in A} Q_w^*(s', a')]$

Wrong. The min is taken over A , the wizard's own actions, but a rational wizard never minimizes its own reward. This also skips the goblin's turn entirely, treating s' as if the wizard acts next.

☐ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a' \in A} Q_w^*(s', a')]$

Wrong. The goblin's turn is ignored completely. This is just standard single agent Bellman.

☐ $Q_w^*(s, a) = \sum_{s'} R(s, a, s') + \gamma \max_{a' \in A} Q_w^*(s', a')$

Wrong. The transition probability $T(s, a, s')$ is missing as a weight inside the sum, so this is not a valid expected value, each s' is counted equally regardless of how likely it is.

☒ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \min_{b \in B} \sum_{s''} (T(s', b, s'') [R(s', b, s'') + \gamma \max_{a' \in A} Q_w^*(s'', a')])]$

Correct. After the wizard lands in s' , the adversarial goblin picks $b \in B$ to minimize the wizard's future reward ($\min_{b \in B}$), transitioning to s'' (consider the goblin reward is being added to the wizard reward so the goblin is attempting to minimize the value its adding/giving to the wizard). The wizard then acts optimally from s'' ($\max_{a' \in A}$). Both turns, both transition functions, and both rewards are properly accounted for.

☐ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{b \in B} \sum_{s''} (T(s', b, s'') [R(s', b, s'') + \gamma \min_{a' \in A} Q_w^*(s'', a')])]$

Wrong. The goblin maximizes ($\max_{b \in B}$) when it should minimize as the enemy, and the wizard minimizes its own reward ($\min_{a' \in A}$) when it should maximize. Both operators are flipped.

☐ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{b \in B} \sum_{s''} (T(s', b, s'') [R(s', b, s'') + \gamma \max_{a' \in A} Q_w^*(s'', a')])]$

Wrong. The goblin uses $\max_{b \in B}$, which would be correct for a *cooperative* goblin, not an adversarial one. The adversarial goblin should minimize.

☐ $Q_w^*(s, a) =$ _____

(b) (0.3 points) Now, assume that the goblin's heart has grown three sizes and now has decided to become friendly with the goal of maximizing your utility. If we assume both the wizard and the goblin are rational and act optimally according to their new goals, which, if any, of the following expressions correctly applies the Bellman equation to update Q values of wizard actions ($Q_w^*(s, a)$ being the value of the wizard taking action a in state s).

☐ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \min_{a' \in A} Q_w^*(s', a')]$

Wrong. Minimizes over the wizard's own actions and skips the goblin's turn entirely.

☐ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a' \in A} Q_w^*(s', a')]$

Wrong. The goblin's turn is skipped entirely. This models a single-agent MDP with no second player.

☐ $Q_w^*(s, a) = \sum_{s'} R(s, a, s') + \gamma \max_{a' \in A} Q_w^*(s', a')$

Wrong. Missing $T(s, a, s')$ as a weight, so the sum is not a proper expectation over next states.

☐ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \min_{b \in B} \sum_{s''} (T(s', b, s'') [R(s', b, s'') + \gamma \max_{a' \in A} Q_w^*(s'', a')])]$

Wrong. The goblin uses $\min_{b \in B}$, meaning it is still acting adversarially. A friendly goblin that wants to maximize the wizard's reward should use $\max_{b \in B}$.

☐ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{b \in B} \sum_{s''} (T(s', b, s'') [R(s', b, s'') + \gamma \min_{a' \in A} Q_w^*(s'', a')])]$

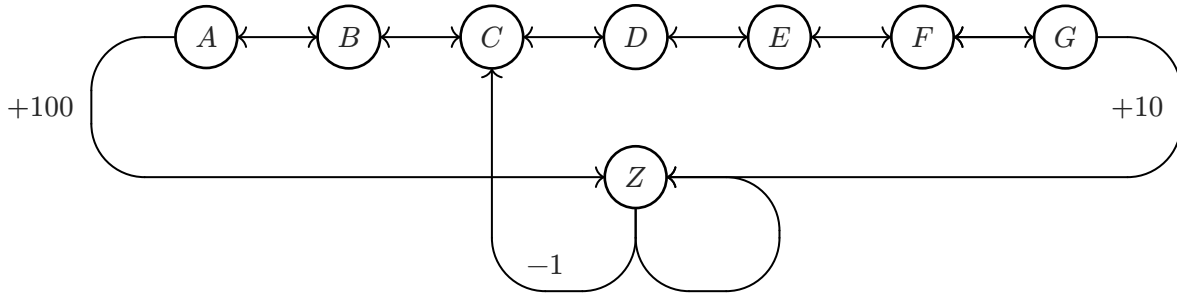
Wrong. The goblin correctly uses $\max_{b \in B}$, but the wizard then minimizes its own reward ($\min_{a' \in A}$), which is never rational.

☒ $Q_w^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{b \in B} \sum_{s''} (T(s', b, s'') [R(s', b, s'') + \gamma \max_{a' \in A} Q_w^*(s'', a')])]$

Correct. The friendly goblin picks $b \in B$ to maximize the wizard's reward ($\max_{b \in B}$), and the wizard then also acts to maximize from s'' ($\max_{a' \in A}$). Both agents are cooperating, so both use max.

☐ $Q_w^*(s, a) =$ _____

5. (1.3 points) Consider an infinite horizon MDP with deterministic transitions on actions $\{\leftarrow, \rightarrow\}$, over the network shown below with zero reward at all transitions unless otherwise labeled.



(a) (0.3 points) What is the *smallest* discount factor γ_m , such that the optimal policy may be $\pi^*(Z) = \leftarrow$

$$\gamma_m = (1/100)^{1/3} = 1/\sqrt[3]{100}$$

At the threshold, $\pi^*(Z) = \rightarrow$ (stay) gives $V^*(Z) = 0$.

For $\leftarrow$ to be equally good: $Q(Z, \leftarrow) = -1 + \gamma \cdot V^*(C) = 0$, so $V^*(C) = 1/\gamma$.

Under the $\leftarrow$ policy from C , we get $V^*(C) = 100\gamma^2$ (two steps to A , then $+100$, with $V^*(Z) = 0$).

Setting equal: $100\gamma^3 = 1 \rightarrow \gamma_m = (1/100)^{1/3}$.

(b) (0.2 points) In terms of the discount factor γ_m from the previous question, consider running value iteration.

What is the smallest iteration number k (starting from initial values at $k = 0$ and results after a single iteration at $k = 1$), where $V_k(C) > 0$. For this iteration, what is the value of $V_k(C)$.

$$k = 3 \quad V_k(C) = V_3(C) = \sqrt[3]{100}$$

The $+100$ reward is two hops from C (via B then A).

Value propagates outward one step per iteration: $k = 1$ reaches A ($V_1(A) = 100$), $k = 2$ reaches B ($V_2(B) = 100\gamma_m$), $k = 3$ reaches C :

$$V_3(C) = \gamma_m \cdot V_2(B) = 100\gamma_m^2 = 100 \cdot (1/100)^{2/3} = 100^{1/3} = \sqrt[3]{100}.$$

(c) (0.2 points) Similarly, what is the smallest iteration number k , where $V_k(F) > 0$. For this iteration, what is the value of $V_k(F)$.

$$k = 2 \quad V_k(F) = \sqrt[3]{10}$$

F is one hop from G . At $k = 1$, $V_1(G) = 10$. At $k = 2$: $V_2(F) = \gamma_m \cdot V_1(G) = 10\gamma_m = 10 \cdot (1/100)^{1/3} = 10^{1/3} = \sqrt[3]{10}$.

(d) (0.3 points) Again using the discount factor γ_m , what is the optimal policy π^* ?

$\pi^*(A)$	$\pi^*(B)$	$\pi^*(C)$	$\pi^*(D)$	$\pi^*(E)$	$\pi^*(F)$	$\pi^*(G)$	$\pi^*(Z)$
←	←	←	←	→	→	→	←

The indifference boundary falls at chain position $p = 3.75$ (between D and E). Label the chain $A = 0, B = 1, \dots, G = 6$. From position p , going left collects +100 at step p (value $100\gamma^p$); going right collects +10 at step $6 - p$ (value $10\gamma^{6-p}$). Setting equal: $100\gamma^p = 10\gamma^{6-p} \rightarrow \gamma^{6-2p} = 10$. At γ_m this gives $p = 3.75$, so A, B, C, D ($p \leq 3$) go ← and E, F, G ($p \geq 4$) go →. Z is at indifference and takes ←.

(e) (0.3 points) If we decide to increase γ , as γ approaches 1, what does the optimal policy converge to?

$\pi^*(A)$	$\pi^*(B)$	$\pi^*(C)$	$\pi^*(D)$	$\pi^*(E)$	$\pi^*(F)$	$\pi^*(G)$	$\pi^*(Z)$
←	←	←	←	←	←	←	←

As $\gamma \rightarrow 1$ discounting disappears and the agent weights all future rewards equally. Since $+100 > +10$ regardless of distance, every state prefers heading toward A . The boundary $\gamma^{6-2p} = 10$ requires $6 - 2p \rightarrow -\infty$ as $\gamma \rightarrow 1$, pushing the crossover point past G , so all states converge to ←.

6. (1 points) After fighting goblins all quarter the robot wizard decides to have a fun summer break and go to a water park. At the park are multiple ladders set up in sequence to progressively higher slides. The slides are fun and give a reward of +32, but the ladders are tough to climb and have a reward of -2 , and are also slippery with a chance of falling (which we can model as a single state where we need to call the robot paramedics) with a reward of -128 . The problem is that the wizard does not know the probability of falling, and so is unsure if it should risk the ladder. Luckily, it can use reinforcement learning to find out. For this problem we may assume a discount of $\gamma = 1$, and we will use the following two episodes (sequences of samples of the form (s, a, s', r)):

Episode 1:

(Ground, Climb, Platform 1, -2)
 (Platform 1, Climb, Platform 2, -2)
 (Platform 2, Slide, Pool, $+32$)

Episode 2:

(Ground, Climb, Platform 1, -2)
 (Platform 1, Climb, Fall, -128)

(a) (0.3 points) What are the values of the states: Ground, Platform 1, and Platform 2, after performing Temporal-Difference Learning with a learning rate of $\alpha = 0.5$ using *only episode 1*.

TD update: $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$. All values start at 0, $\gamma = 1$, $\alpha = 0.5$.

$V(\text{Ground}) = (G, C, P_1, -2): V(G) \leftarrow 0 + 0.5[-2 + 0 - 0] = -1$

$V(\text{Platform 1}) = (P_1, C, P_2, -2): V(P_1) \leftarrow 0 + 0.5[-2 + 0 - 0] = -1$

$V(\text{Platform 2}) = (P_2, S, \text{Pool}, +32): V(P_2) \leftarrow 0 + 0.5[32 + 0 - 0] = 16$

(b) (0.3 points) What are the values of the states: Ground, Platform 1, and Platform 2, after performing Temporal-Difference Learning with a learning rate of $\alpha = 0.5$ using *episode 1 followed by episode 2*.

Starting from Episode 1 values ($V(G) = -1$, $V(P_1) = -1$, $V(P_2) = 16$, $V(\text{Fall}) = 0$):

$V(\text{Ground}) = (G, C, P_1, -2): V(G) \leftarrow -1 + 0.5[-2 + (-1) - (-1)] = -1 + 0.5(-2) = -2$

$V(\text{Platform 1}) = (P_1, C, \text{Fall}, -128): V(P_1) \leftarrow -1 + 0.5[-128 + 0 - (-1)] = -1 + 0.5(-127) = -64.5$

$V(\text{Platform 2}) = V(\text{Platform 2}) = 16$ unchanged (not visited in Episode 2)

(c) (0.3 points) What are the values of the following state/action pairs after performing Q-Learning with a learning rate of $\alpha = 0.5$ using *episode 1 followed by episode 2*.

Q-Learning update: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \max_{a'} Q(s', a') - Q(s, a)]$. All Q values start at 0.

Episode 1:

- $(G, C, P_1, -2): Q(G, C) \leftarrow 0 + 0.5[-2 + 0] = -1$
- $(P_1, C, P_2, -2): Q(P_1, C) \leftarrow 0 + 0.5[-2 + 0] = -1$
- $(P_2, S, \text{Pool}, +32): Q(P_2, S) \leftarrow 0 + 0.5[32 + 0] = 16$

Episode 2 (note: $\max Q(P_1, \cdot) = \max(Q(P_1, \text{Climb}), Q(P_1, \text{Slide})) = \max(-1, 0) = 0$):

- $(G, C, P_1, -2): Q(G, C) \leftarrow -1 + 0.5[-2 + 0 - (-1)] = -1 + 0.5(-1) = -1.5$
- $(P_1, C, \text{Fall}, -128): Q(P_1, C) \leftarrow -1 + 0.5[-128 + 0 - (-1)] = -1 + 0.5(-127) = -64.5$

$Q(\text{Ground}, \text{Climb}) = -1.5$

$Q(\text{Platform 1}, \text{Slide}) = 0$

$Q(\text{Platform 1}, \text{Climb}) = -64.5$

$Q(\text{Platform 2}, \text{Slide}) = 16$

(d) (0.1 points) Were the methods used in this problem best described as *model based* or *model free*.

☐ *model based*

☒ *model free*

TD learning and Q-learning both update directly from sampled (s, a, s', r) experience without building an explicit transition model $T(s, a, s')$.¹

7. (0.2 points) Finally, for guaranteed points, please try your best to draw your version of the robot wizard (maybe even on the water-slide!)



shutterstock.com • 2727939559

Figure 3: Have a great summer everyone!

¹Fun Fact! In 2016, an OpenAI agent trained model free on the boat racing game CoastRunners discovered it could outscore every human by driving in circles, catching fire, and respawning to get points(despite never finishing a lap). RL learning is very tricky :) . <https://openai.com/index/faulty-reward-functions/>